

Peeking at the Hessian: HVPs, Lanczos, and Stochastic Quadrature

Samyak Jain

How to learn about the eigenvalues of a matrix you can never write down.

Motivation: The Hessian of a neural network’s loss is a $p \times p$ matrix, where p is the number of parameters. For a model with 100 million parameters that is about *40 petabytes* in float32, which is almost impossible to compute in closed form. But its spectrum can capture a lot of rich structure which influences the learning dynamics. Therefore, understanding the eigenspectrum of the hessian is important.

The good news is that there are tools which prevent us from computing the closed form $p \times p$ version of the hessian, yet we can understand its eigenspectrum. Everything in this doc is built from one primitive, the **Hessian-vector product**, which costs less than three gradient iterations. In this blogpost, we walk up the ladder of algorithms aiming to estimate the eigenvalues, statistics of eigenvalues like trace and eigenspectrum. At the end we play around with an [interactive widget](#) to build a deep intuition for each one, on random matrices and on real (small) neural net Hessians.

Contents

1	Setup and notation	2
2	Hessian-vector products and the cost of autodiff	2
2.1	A backward pass costs twice a forward pass	2
2.2	The Hessian-vector product trick	2
2.3	Compute of the Full Hessian	3
3	Power iteration	3
4	Lanczos iteration	4
4.1	Orthogonalizing the Krylov basis	4
4.2	From T to eigenvalues and eigenvectors of A	5
5	Block Lanczos	5
6	The Hutchinson trace estimator	6
7	Stochastic Lanczos Quadrature (SLQ)	7
7.1	Estimating the quadratic form with Lanczos	7
7.2	Predicting the spectrum	8
7.3	Further variants of SLQ	8

8	The Kernel Polynomial Method (KPM)	8
9	Summary	9
10	The interactive widget	10

1 Setup and notation

Throughout, we work with a loss $L(w) = \frac{1}{n} \sum_{i=1}^n \ell(w; x_i)$ over

- n : number of training samples,
- p : number of parameters,

and we care about the (symmetric) Hessian $H = \nabla^2 L(w) \in \mathbb{R}^{p \times p}$, with eigendecomposition $H = \sum_i \lambda_i u_i u_i^\top$, eigenvalues ordered $|\lambda_1| \geq |\lambda_2| \geq \dots$. Most of what follows holds for any symmetric matrix. We write A when the discussion is generic and H when the Hessian cost model matters.

For compute counts we use the standard dense network: a forward pass costs roughly np operations, since every parameter does one multiply and accumulate per sample.

2 Hessian-vector products and the cost of autodiff

2.1 A backward pass costs twice a forward pass

Take a deep linear chain $x \rightarrow W_1 \rightarrow W_2 \rightarrow \dots \rightarrow f(x)$, with activations h_i . The forward pass is one matrix product per layer, $h_i = W_i h_{i-1}$, so about np operations total.

At each layer, backprop has to compute *two* things instead of one:

$$\underbrace{\frac{\partial h_i}{\partial W_i}}_{\text{gradient for the weights}} \quad \text{and} \quad \underbrace{\frac{\partial h_i}{\partial h_{i-1}}}_{\text{signal passed to the previous layer}}$$

Each is a matrix product of the same size as the forward one, so the backward pass costs about $2np$.

2.2 The Hessian-vector product trick

We want $Hv = \nabla^2 L(w)v$ for an arbitrary vector v , without forming H . The key identity is

$$Hv = \nabla_w (\nabla_w L(w)^\top v),$$

i.e. the gradient of the scalar $g(w) = \nabla L(w)^\top v$. Autodiff computes this by differentiating *through the gradient's own computational graph*. A clean way to think about it: the graph that computes the gradient is itself a network whose “layers” are the local Jacobians

$$W_i \rightarrow \frac{\partial h_i}{\partial W_i}, \quad h_i \rightarrow \frac{\partial h_{i+1}}{\partial h_i},$$

and running one more backward pass through that graph. Think of it as replacing the original network with a modified one, $W_i \rightarrow \partial h_i / \partial W_i$ and $h_i \rightarrow \partial h_{i+1} / \partial h_i$. Now everything is the same as before, one level up. Thus we have:

$$\underbrace{np}_{\text{forward}} + \underbrace{2np}_{\text{1st backward}} + \underbrace{2np}_{\text{2nd backward}} = 5np.$$

So **one HVP costs about $5np$** , less than three gradients. The rule of thumb: *an HVP is a small constant times a gradient*. Memory is $O(p)$ on top of activation storage. One practical caveat: double backward has to retain the first backward's graph for the second pass, which roughly doubles activation memory.

2.3 Compute of the Full Hessian

To materialize H we can run p HVPs, one per basis vector. This leads to the following cost:

$$np + 2np + 2np \cdot p = 3np + 2np^2,$$

plus $O(p^2)$ memory just to store the thing. Both are hopeless for any modern network. So the magic of hessian vector product really makes things very much practical.

Operation	Compute	Memory (beyond activations)
Forward pass	np	$O(p)$
Backward pass (gradient)	$2np$	$O(p)$
Hessian-vector product	$5np$	$O(p)$
Full Hessian	$3np + 2np^2$	$O(p^2)$

Our main goal now is to understand the structure of eigenvalues of large symmetric matrices while only ever touching the first three rows of this table. For this we dive into power iteration, then Lanczos, block Lanczos, and the stochastic estimators: Hutchinson, stochastic Lanczos quadrature, and the kernel polynomial method.

3 Power iteration

The simplest question: what is the top eigenvalue λ_1 and its eigenvector u_1 ?

Take a random vector x and repeatedly apply A , normalizing as you go. Expanding x in the eigenbasis, $x = \sum_i (u_i^\top x) u_i$, after k applications:

$$\frac{A^k x}{\|A^k x\|} = \frac{\sum_i \lambda_i^k (u_i^\top x) u_i}{(\sum_i \lambda_i^{2k} (u_i^\top x)^2)^{1/2}} \xrightarrow{k \rightarrow \infty} \pm u_1,$$

because $(\lambda_i / \lambda_1)^k \rightarrow 0$ for every $i > 1$. The $\pm$ is per step: if $\lambda_1 < 0$ the normalized iterate flips sign every iteration, so only the line $\text{span}\{u_1\}$ converges. The error decays geometrically at rate $|\lambda_2 / \lambda_1|$, so the closer the second eigenvalue is to the first, the slower the convergence.

1: $x \leftarrow$ random unit vector in $\mathbb{R}^p$

2: **for** $t = 1, \dots, k$ **do**

3: $x \leftarrow Ax$

▷ one HVP if $A = H$: $5np$

4: $x \leftarrow x/\|x\|$
5: **end for**
6: **return** $\hat{\lambda} = x^\top Ax$ (eigenvalue estimation)

Cost. k matrix-vector products. For an explicitly stored dense $\sqrt{p} \times \sqrt{p}$ matrix each matvec costs $O(p)$, so this is $O(kp)$. For the Hessian accessed through HVPs it is $O(k \cdot 5np) = O(knp)$. The p^2 never appears.

That is all power iteration is. It gives one eigenpair, and it wastes information: it computes the whole Krylov sequence $x, Ax, A^2x, \dots$ and then throws away everything except the last vector. We will see how lanczos keeps it and this helps in computing multiple top eigenvalues simultaneously.

4 Lanczos iteration

Key idea. Instead of keeping only $A^k x$, keep the whole subspace it swept through, the *Krylov subspace*

$$\mathcal{K}_k = \text{span}\{x, Ax, A^2x, \dots, A^{k-1}x\}.$$

Intuitively this is the subspace in which A “acts the most” starting from a random direction. Project A into $\mathcal{K}_k$ and the top eigenvalues of the small $k \times k$ projected matrix are excellent approximations to the extreme eigenvalues of A . From those we can also recover eigenvector estimates.

4.1 Orthogonalizing the Krylov basis

The raw vectors $v_1 = x, v_2 = Ax, \dots, v_k = A^{k-1}x$ are a terrible basis: they all lean toward u_1 , which is exactly what power iteration exploits. So we run Gram–Schmidt, starting from v_1 :

$$\tilde{v}_1 = v_1, \quad \tilde{v}_j = v_j - \sum_{i < j} (v_j^\top \tilde{v}_i) \tilde{v}_i \quad (\text{then normalize}).$$

Collect the orthonormal basis as $\tilde{V} = [\tilde{v}_1 \ \tilde{v}_2 \ \dots \ \tilde{v}_k] \in \mathbb{R}^{p \times k}$.

Claim (the projected matrix is tridiagonal). *If A is symmetric, then $T = \tilde{V}^\top A \tilde{V}$ is **tridiagonal**: $\tilde{v}_j^\top A \tilde{v}_i = 0$ whenever $|j - i| > 1$.*

Proof sketch. By construction $\tilde{v}_j \in \mathcal{K}_j$, so $A\tilde{v}_j \in \mathcal{K}_{j+1} = \text{span}\{\tilde{v}_1, \dots, \tilde{v}_{j+1}\}$. For any $\ell > 1$, the vector $\tilde{v}_{j+\ell}$ is orthogonal to all of $\mathcal{K}_{j+1}$, because Gram–Schmidt made it so. Hence

$$\tilde{v}_{j+\ell}^\top A \tilde{v}_j = 0 \quad \forall \ell > 1,$$

which kills the lower triangle below the first subdiagonal. Symmetry of A transfers the same statement to the upper triangle: $\tilde{v}_j^\top A \tilde{v}_{j+\ell} = (\tilde{v}_{j+\ell}^\top A \tilde{v}_j)^\top = 0$. $\square$

So the projection has the form

$$T = \tilde{V}^\top A \tilde{V} = \begin{pmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \beta_2 & \\ & \beta_2 & \ddots & \beta_{k-1} \\ & & \beta_{k-1} & \alpha_k \end{pmatrix}, \quad \alpha_i = \tilde{v}_i^\top A \tilde{v}_i, \quad \beta_i = \tilde{v}_{i+1}^\top A \tilde{v}_i.$$

While this way is clean and gives an intuitive picture, in practice a more complicated version of Lanczos is used, which will give rise to the exact same T as what we just described, but each steps are different. Basically, if we read off column j , we will get: **three-term recurrence**

$$\beta_j \tilde{v}_{j+1} = A \tilde{v}_j - \alpha_j \tilde{v}_j - \beta_{j-1} \tilde{v}_{j-1},$$

so in exact arithmetic each new basis vector needs orthogonalization against only the *two* previous ones, not all of them. This is cheaper and easier to implement, though in floating point orthogonality still degrades, which is why practical codes re-orthogonalize. Thus the lanczos iteration, which one would find in most places is the following:

```

1:  $v_1 \leftarrow z/\|z\|$ ;  $\beta_0 \leftarrow 0$ ;  $v_0 \leftarrow 0$ 
2: for  $j = 1, \dots, k$  do
3:    $w \leftarrow A v_j$  ▷ one HVP:  $5np$ 
4:    $\alpha_j \leftarrow v_j^\top w$ 
5:    $w \leftarrow w - \alpha_j v_j - \beta_{j-1} v_{j-1}$ 
6:    $\beta_j \leftarrow \|w\|$ ;  $v_{j+1} \leftarrow w/\beta_j$ 
7: end for
8:  $T \leftarrow \text{tridiag}(\beta; \alpha; \beta)$ 
```

4.2 From T to eigenvalues and eigenvectors of A

Diagonalize the small matrix: $T = S \Theta S^\top$ with $\Theta = \text{diag}(\theta_1, \dots, \theta_k)$. If $k = p$ (full run), $\tilde{V}$ is square and orthogonal, so $\tilde{V}^\top A \tilde{V} = T$ rearranges to

$$A = \tilde{V} T \tilde{V}^\top = (\tilde{V} S) \Theta (\tilde{V} S)^\top,$$

an exact eigendecomposition. For $k \ll p$ the same objects are approximations, which are called **Ritz values** θ_i and **Ritz vectors** $y_i = \tilde{V} s_i$. A useful check on quality is the residual

$$\text{error} = \sum_i \|A(\tilde{V} s_i) - \theta_i(\tilde{V} s_i)\|.$$

Cost. For the Hessian: k HVPs = $O(knp)$, plus $O(k^2 p)$ if we fully reorthogonalize, plus $O(k^2)$ to $O(k^3)$ for the eigendecomposition of the tridiagonal T (negligible: k is tens, p is millions), plus $O(kp)$ per Ritz vector we choose to form. The dominant term in deep learning settings is always the $O(knp)$ of the HVPs.

5 Block Lanczos

Plain Lanczos has a blind spot: **eigenvalue multiplicity**. A single Krylov sequence started from one vector can only ever see a one-dimensional slice of each eigenspace, so eigenvalues that are exactly degenerate (or clustered so tightly that floating point cannot separate them) are found once, not with their multiplicity. Lanczos simply cannot tell them apart.

Key idea. Do power iteration on a rank- b subspace instead of a rank-one subspace: start from a random $p \times b$ block Q and build the block Krylov space from $Q, AQ, A^2 Q, \dots$. A block of width b can capture up to b copies of a degenerate eigenvalue and cleanly separates tight clusters.

It is the same as Lanczos iteration, but with Gram–Schmidt orthogonalization at two levels:

- *between* iterates: each new block only needs orthogonalization against the previous *two* blocks, since block tridiagonality gives the same three-term recurrence with $b \times b$ coefficient blocks.
- *within* an iterate: the b columns of each block are orthogonalized against each other (a QR decomposition).

The projected matrix is then **block tridiagonal**: the same structure with $b \times b$ blocks in place of scalars. Concretely, with $Q_j \in \mathbb{R}^{p \times b}$ the j -th orthonormal block, the three-term recurrence becomes

$$AQ_j = Q_{j-1}B_{j-1}^\top + Q_j A_j + Q_{j+1}B_j, \quad A_j = Q_j^\top A Q_j, \quad B_j \text{ from QR of the residual } W_j = AQ_j - Q_j A_j - Q_{j-1}B_{j-1}^\top$$

so T has the A_j on its block diagonal and the B_j on the block off-diagonals, and the eigendecomposition of the small $kb \times kb$ matrix T supplies the Ritz pairs exactly as before.

Cost. Per iteration, applying A to the $p \times b$ block is b matvecs, so $b \cdot 5np = O(npb)$ via HVPs. Orthogonalization against the two previous blocks plus the within-block QR adds $O(kpb^2)$ total ($O(k^2pb^2)$ with full reorthogonalization), and the eigendecomposition of the $kb \times kb$ block tridiagonal matrix costs $O((kb)^2)$ to $O((kb)^3)$. The HVP term $O(knpb)$ dominates the orthogonalization when $n \gg kb$ (the p 's cancel).

Everything so far targets *individual extreme eigenpairs*. The remaining three methods answer *global* questions about the spectrum (traces, densities, counts) using randomness.

6 The Hutchinson trace estimator

How do we estimate $\text{Tr}(A) = \sum_i \lambda_i$ when you can only touch A through matvecs? Draw a random probe z with $\mathbb{E}[zz^\top] = I$ (e.g. $z \sim \mathcal{N}(0, I)$). Then

$$\mathbb{E}[z^\top A z] = \mathbb{E}[\text{Tr}(A z z^\top)] = \text{Tr}(A \mathbb{E}[z z^\top]) = \text{Tr}(A).$$

So the Monte Carlo estimator with s probes is

$$\text{Tr}(A) \approx \frac{1}{s} \sum_{i=1}^s z_i^\top A z_i,$$

costing s matvecs, so $O(snp)$ for the Hessian. No p^2 anywhere.

Remark (why Rademacher probes). *In practice z is usually drawn with i.i.d. Rademacher entries (± 1 with probability $\frac{1}{2}$) rather than Gaussian. The diagonal of $zz^\top$ is then exactly 1, so no randomness is left on the diagonal: the estimator is exact on the diagonal contribution and only the off-diagonal terms add noise. Concretely, $\text{Var}(z^\top A z) = 2 \sum_{i \neq j} A_{ij}^2 = 2(\|A\|_F^2 - \sum_i A_{ii}^2)$ for Rademacher vs $2\|A\|_F^2$ for Gaussian. The convergence of $\frac{1}{s} \sum z_i z_i^\top \rightarrow I$ is faster in case of rademacher probes.*

7 Stochastic Lanczos Quadrature (SLQ)

Hutchinson estimates $\text{Tr}(A)$. SLQ generalizes it to **any spectral function**:

$$\text{Tr}(f(A)) = \sum_i f(\lambda_i) = \mathbb{E}[z^\top f(A) z] \approx \frac{1}{s} \sum_{i=1}^s z_i^\top f(A) z_i.$$

This covers a lot: $f(t) = \log t$ gives the log determinant, $f(t) = t^2$ gives $\|A\|_F^2$, $f(t) = \mathbf{1}(t \leq c)$ counts eigenvalues below a threshold, and $f(t) = \delta_\sigma(t - t_0)$ reads out the *spectral density* near t_0 .

The only hard part is the quadratic form $z^\top f(A) z$. The key secret sauce of SLQ is to approximate

$$z^\top f(A) z \approx \|z\|^2 \sum_{j=1}^k \tau_j f(\theta_j), \quad \tau_j = (S_{1j})^2,$$

where θ_j and S come from k Lanczos steps on A itself, as we explain next.

7.1 Estimating the quadratic form with Lanczos

Fix one probe $z \sim \mathcal{N}(0, I)$ and normalize it, $v_1 = z/\|z\|$. Let $B = f(A)$. What we want is the quadratic form $v_1^\top B v_1$. Since B has the same eigenvectors as A with eigenvalues $f(\lambda_i)$,

$$v_1^\top B v_1 = \sum_{i=1}^p (u_i^\top v_1)^2 f(\lambda_i).$$

So the exact answer is a weighted sum of f over the eigenvalues of A , with weights $(u_i^\top v_1)^2$ that sum to one. We cannot afford all p weights, and we only have matvecs with A , not with B . But it turns out a plain Lanczos run on A is enough:

To see why this works, expand v_1 in the eigenbasis of A . Powers act on eigenvalues,

$$A^n v_1 = \sum_i \lambda_i^n (u_i^\top v_1) u_i,$$

so for any f written in its polynomial basis,

$$f(A) v_1 = \sum_i f(\lambda_i) (u_i^\top v_1) u_i \quad \implies \quad v_1^\top f(A) v_1 = \sum_i f(\lambda_i) (u_i^\top v_1)^2.$$

Note that the Lanczos run hands us exactly these quantities: the Ritz values θ_j stand in for the eigenvalues λ_i , and $(S_{1j})^2$ stands in for the combined weight $(u_i^\top v_1)^2$ of the eigenvalues near θ_j . Thus repeating the process over multiple probes gives estimates of the form $\text{Tr}(f(A))$, exactly like Hutchinson:

- 1: **for** $i = 1, \dots, s$ **do**
- 2: $z \sim \mathcal{N}(0, I)$ (or Rademacher); $v_1 \leftarrow z/\|z\|$
- 3: run k Lanczos steps on A from $v_1 \rightarrow T^{(i)}$ $\triangleright k$ HVPs
- 4: diagonalize $T^{(i)} = S\Theta S^\top$
- 5: $q^{(i)} \leftarrow \|z\|^2 \cdot \sum_j (S_{1j})^2 f(\theta_j)$
- 6: **end for**
- 7: **return** $\frac{1}{s} \sum_i q^{(i)}$

7.2 Predicting the spectrum

The empirical spectral density $\rho(t) = \frac{1}{p} \sum_i \delta(t - \lambda_i)$ is the special case where f is a delta at t (in practice a narrow Gaussian bump is used instead). Basically, $(u_i^\top v_1)^2$ is the weight the probe places on eigenvalue λ_i ; the quadrature estimates these weights at the k Ritz values, and some Gaussian smoothing then turns the weighted nodes into an eigenspectrum:

$$\hat{\rho}(t) = \frac{1}{s} \sum_{i=1}^s \sum_{j=1}^k \tau_j^{(i)} g_\sigma(t - \theta_j^{(i)}), \quad \tau_j = (S_{1j})^2.$$

This is the estimator behind the Hessian eigenspectrum plots in the deep learning literature (Ghorbani et al., PyHessian).

Time complexity. Let s be the number of Hutchinson probes. SLQ costs $O(s \cdot t)$ where t is the Lanczos iteration cost, i.e. $O(skn p)$ with HVPs.

7.3 Further variants of SLQ

A clear problem with SLQ is that it can saturate over the top eigenvalues multiple times, even if they are not present that frequently: every probe's Krylov space is attracted to the same few outliers, which carry only weight $1/p$ each in the density. There are ways to overcome this:

- a) Orthogonalize in the space of later iterates of Lanczos to determine the dominant subspace, and run SLQ in the remaining space. Block Lanczos can also be used here.
- b) Run Lanczos with starting vectors orthogonal to the space found previously. This can lead to faster convergence onto parts of the spectrum not seen yet.

In summary, the later iterates of Lanczos estimate the top eigenvalues well, but the earlier iterates still find the broader clusters of eigenvalues, which is why SLQ can use Lanczos at all. Tabs 6 to 9 of the widgets implement exactly these variants. Although the above variants seems intuitive but as per the widgets they don't make much of a difference.

8 The Kernel Polynomial Method (KPM)

The Chebyshev polynomials have an interesting property:

$$T_{k+1}(x) = 2x T_k(x) - T_{k-1}(x), \quad T_k(x) = \cos(k \arccos x),$$

and they are orthogonal on $[-1, 1]$ under the measure $1/\sqrt{1-x^2}$:

$$\int_{-1}^1 \frac{T_j(x) T_k(x)}{\sqrt{1-x^2}} dx = \begin{cases} 0 & j \neq k, \\ \pi & j = k = 0, \\ \pi/2 & j = k \geq 1. \end{cases}$$

KPM estimates the spectral density directly in this basis. The basis lives on $[-1, 1]$, so first rescale: estimate $\lambda_{\min}$ and $\lambda_{\max}$ with a short Lanczos run and map

$$\tilde{A} = \frac{2A - (\lambda_{\max} + \lambda_{\min}) I}{\lambda_{\max} - \lambda_{\min}},$$

so the eigenvalues $\tilde{\lambda}_i$ of $\tilde{A}$ land in $[-1, 1]$.

Now write the density in the eigenbasis of Chebyshev polynomials. The general form with measure $1/\sqrt{1-x^2}$ is

$$\tilde{\rho}(x) = \frac{1}{\sqrt{1-x^2}} \sum_{k=0}^{\infty} a_k T_k(x),$$

where $\tilde{\rho}(x) = \frac{1}{p} \sum_i \delta(x - \tilde{\lambda}_i)$. Finding the coefficients a_k using orthogonality (multiply by T_j and integrate), we get the density estimate:

Result (KPM density estimate).

$$\hat{\rho}(x) = \frac{1}{\pi\sqrt{1-x^2}} \left[\hat{\mu}_0 + 2 \sum_{k=1}^K \hat{\mu}_k T_k(x) \right], \quad \mu_k = \frac{1}{p} \text{Tr } T_k(\tilde{A}).$$

Note that KPM approximates each trace $\text{Tr } T_k(\tilde{A})$ using Hutchinson, which requires random probes.

Time complexity. With s Hutchinson probes and K KPM iterations the cost is approximately $O(sKp)$ matvec work; with HVPs it increases to $O(sK \cdot 5np) \approx O(sKnp)$. Same shape as SLQ, with $k \leftrightarrow K$.

9 Summary

All costs assume access to $A = H$ via HVPs at $5np$ each; k = iterations/degree, s = probes, b = block size.

Method	What you get	Compute
Power iteration	top eigenpair (λ_1, u_1)	$O(knp)$
Lanczos	extreme eigenpairs	$O(knp) + O(k^2p)$ reorth.
Block Lanczos	clustered / degenerate extremes	$O(knpb)$
Hutchinson	$\text{Tr}(A)$	$O(snp)$
SLQ	$\text{Tr } f(A)$, spectral density	$O(skn p)$
KPM	spectral density	$O(skn p)$

The through line: a forward pass costs np , an HVP costs $5np$, and every question about the Hessian spectrum, from “what is the sharpness?” to “what does the whole density look like?”, reduces to a few hundred or a few thousand HVPs, arranged cleverly.

References and further reading

- B. Pearlmutter. *Fast exact multiplication by the Hessian*. Neural Computation, 1994. The $O(np)$ HVP.
- G. Golub, G. Meurant. *Matrices, Moments and Quadrature with Applications*. Princeton, 2010. Lanczos as Gauss quadrature.
- M. Hutchinson. *A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines*. 1989.

- S. Ubaru, J. Chen, Y. Saad. *Fast estimation of $\text{tr}(f(A))$ via stochastic Lanczos quadrature*. SIMAX, 2017.
- L. Lin, Y. Saad, C. Yang. *Approximating spectral densities of large matrices*. SIAM Review, 2016. SLQ and KPM for spectral densities, side by side.
- A. Weiße, G. Wellein, A. Alvermann, H. Fehske. *The kernel polynomial method*. Rev. Mod. Phys., 2006.
- B. Ghorbani, S. Krishnan, Y. Xiao. *An investigation into neural net optimization via Hessian eigenvalue density*. ICML 2019. SLQ on deep nets.
- Z. Yao, A. Gholami, K. Keutzer, M. Mahoney. *PyHessian: neural networks through the lens of the Hessian*. 2020.

10 The interactive widget

Finally, let’s play around with some widgets. Everything in this doc is runnable in the [interactive widget](#), in ten numbered estimator tabs.

There are two matrix sources. The first is a random symmetric matrix (GOE, Wishart, spiked, clustered, or log spaced), with the dimension p as a hyperparameter. The second is the exact Hessian of a small one hidden layer MLP, touched only through HVPs; the architecture, n , the activation, and the training are all adjustable. Up to $p = 6000$ the ground truth is the exact eigendecomposition. For larger p the widget uses an SLQ reference spectrum instead, clearly labeled in every chart, so much larger sizes run (up to $p = 8000$ for the random source). Every run also reports how many matvecs it spent.

Each tab shows two main plots. The density plot compares the estimated spectral density with the ground truth. The eigenvalue vs index plot shows λ_i against i , from largest to smallest, which makes it easy to see how well the top of the spectrum is resolved. For the stochastic estimators this curve comes from inverting the estimated density, $\hat{\lambda}_i = \hat{F}^{-1}(1 - \frac{i-1/2}{p})$.

Tabs 6 to 9 use ideas we have not defined yet:

- **Edge + deflated SLQ** (tab 6). First run a few rounds of Lanczos and collect the converged extreme Ritz vectors into an orthonormal U . Then run SLQ on the deflated operator $A' = (I - UU^\top) A (I - UU^\top)$, with probes kept orthogonal to U (write m for the number of harvested eigenpairs, the columns of U). The harvested eigenvalues enter the density exactly, and SLQ fills in the rest:

$$\hat{\rho}(t) = \frac{1}{p} \sum_{\theta \in \text{harvested}} g_\sigma(t - \theta) + \frac{p - m}{p} \hat{\rho}_{\text{SLQ on } A'}(t).$$

- **Orthogonalized probes** (tab 7). Before its own Lanczos run, each new probe is projected off everything the earlier probes already explored. This forces exploration of new directions and costs zero extra matvecs.
- **Block SLQ** (tabs 8 and 9). The probe is a random $p \times b$ block instead of a single vector. Block Lanczos gives a $kb \times kb$ matrix $T = S\Theta S^\top$, and the weights become the squared first block components, $\tau_j = \frac{1}{b} \|S_{1:b,j}\|^2$, which again sum to one. Tab 9 of the widget combines this with orthogonalized probes.